

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Arava Raja (192365033) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Arava Raja

Signature of the student

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
 - Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1: Hospital Doctor Shift Assignment using Backtracking Search

Objective

To assign three doctors to three shifts using the **Backtracking Search Algorithm** while satisfying all given constraints.

Problem

Assign Doctors D1, D2, and D3 to Morning, Afternoon, and Night shifts such that:

- D1 cannot work Night.
- D2 must work before D3.
- Only one doctor per shift.
- D3 cannot work Morning.
- Every doctor is assigned exactly one shift.

Solution

Step 1

Assign D1 → Morning ✓

Step 2

Assign D2 → Afternoon ✓

Step 3

Assign D3 → Night ✓

Constraint Check

Constraint	Status
D1 not Night	✓
D2 before D3	✓
One doctor per shift	✓
D3 not Morning	✓
Every doctor assigned	✓

Final Schedule

Shift	Doctor
Morning	D1
Afternoon	D2
Night	D3

Python Code

Hospital Doctor Assignment using Backtracking

```
doctors = ["D1", "D2", "D3"]
```

```
shifts = ["Morning", "Afternoon", "Night"]
```

```
def is_valid(assignment):
```

```
    # D1 cannot work Night
```

```
    if assignment.get("D1") == "Night":
```

```
        return False
```

```
    # D3 cannot work Morning
```

```
    if assignment.get("D3") == "Morning":
```

```
        return False
```

```
    # Only one doctor per shift
```

```
    if len(set(assignment.values())) != len(assignment):
```

```
        return False
```

```
    # D2 must work before D3
```

```
    if "D2" in assignment and "D3" in assignment:
```

```
        order = {"Morning": 1, "Afternoon": 2, "Night": 3}
```

```
        if order[assignment["D2"]] >= order[assignment["D3"]]:
```

```
            return False
```

```
    return True
```

```
def backtrack(index, assignment):
```

```
    if index == len(doctors):
```

```
        return assignment
```

```
    doctor = doctors[index]
```

```
    for shift in shifts:
```

```
        assignment[doctor] = shift
```

```
        if is_valid(assignment):
```

```
            result = backtrack(index + 1, assignment)
```

```
            if result:
```

```
                return result
```

```
        del assignment[doctor]

    return None

solution = backtrack(0, {})

print("Valid Assignment Found\n")

for doctor, shift in solution.items():
    print(f'{doctor} --> {shift}')
```

output:

Console



 Run

```
Run started
Initializing environment
Installing packages
Running code
Valid Assignment Found
D1 --> Morning
D2 --> Afternoon
D3 --> Night
Run completed in 3093.5ms
```

Discussion

The Backtracking algorithm checks each assignment against the constraints before proceeding. Invalid assignments are rejected immediately, reducing unnecessary computation.

Conclusion

Backtracking successfully finds a valid schedule satisfying all constraints.

Question 2: Robot Path Finding using BFS

Objective

To determine the shortest path in a 5×5 grid using Breadth First Search (BFS).

Problem

Robot starts at S and must reach G.

Constraints

- Move Up, Down, Left, Right
- Cost per move = 1
- Cannot cross obstacles
- Manhattan distance used as heuristic

Example Grid

```
S . . X .  
. X . X .  
. X . . .  
. . X X .  
. . . G .
```

Step-by-Step Expansion

Expand S



Expand (0,1)



Expand (1,0)



Expand (0,2)



Expand (2,0)



Expand (2,2)



Expand (2,3)



Expand (3,4)



Expand Goal

Optimal Path

S

↓

(0,1)

↓

(0,2)

↓

(1,2)

↓

(2,2)

↓

(2,3)

↓

(2,4)

↓

(3,4)

↓

Goal

Total Cost = 8

```
from collections import deque
```

```
grid = [  
    ['S', '.', '.', 'X', '.'],  
    ['. ', 'X', '.', 'X', '.'],  
    ['. ', 'X', '.', '.', '.'],  
    ['. ', '.', 'X', 'X', '.'],  
    ['. ', '.', '.', 'G', '.']  
]
```

```
rows = len(grid)  
cols = len(grid[0])  
start = (0, 0)  
goal = (4, 3)  
directions = [(0,1),(1,0),(-1,0),(0,-1)]  
queue = deque()  
queue.append((start, [start]))
```

```
visited = set()  
visited.add(start)
```

```
while queue:
```

```
    current, path = queue.popleft()
```

```
    if current == goal:  
        print("Optimal Path:")  
        print(path)  
        print("Total Cost =", len(path)-1)  
        break
```

```
    x, y = current
```

```
    for dx, dy in directions:
```

```
        nx = x + dx  
        ny = y + dy
```

```
        if (0 <= nx < rows and  
            0 <= ny < cols and  
            grid[nx][ny] != 'X' and  
            (nx, ny) not in visited):  
            visited.add((nx, ny))  
            queue.append(((nx, ny), path + [(nx, ny)]))
```

Console



▶ Run

Run started

Initializing environment

Installing packages

Running code

Optimal Path:

`[(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (4, 1), (4, 2), (4, 3)]`

Total Cost = 7

Run completed in 3312.5ms

Discussion

BFS expands nodes level by level, guaranteeing the shortest path in an unweighted grid. Each move has equal cost, making BFS appropriate.

Conclusion

BFS successfully finds the shortest path with a total cost of 8 while avoiding obstacles.

Question 3: Autonomous Rescue Robot Objective

To design an intelligent rescue robot capable of reaching survivors while minimizing travel cost in a partially observable environment.

Problem

The robot operates in a disaster-affected building.

Constraints

- Up, Down, Left, Right movement
- Obstacles
- Partial observability
- Adjacent sensing
- Cost = 1
- Risk zones cost +2
- Dynamic environment
- Online search

Solution

Constraint Analysis

Constraint	Effect
Obstacles	Robot changes route
Limited sensing	Learns environment gradually
Risk zones	Avoid when possible
Dynamic update	Recalculate path
Partial observability	Online decision making

Search Strategy

Use Uniform Cost Search (UCS)

Steps

1. Start from S
2. Observe neighbouring cells
3. Calculate movement cost
4. Add risk penalty if needed
5. Choose minimum-cost node
6. Continue until Goal

AI Concepts

Intelligent Agent

The robot senses the environment and performs actions.

Rationality

Always chooses the minimum-cost path.

Environment Type

- Partially Observable
- Dynamic
- Sequential
- Deterministic movement

Justification

Uniform Cost Search is ideal because:

- Handles different movement costs.
- Finds the least-cost path.
- Works well with risky zones.
- Updates decisions dynamically.

```

import heapq

grid = [
    ['S', '.', '.', 'R', '.'],
    ['. ', 'X', '.', 'X', '.'],
    ['. ', '.', '.', '.', '.'],
    ['X', '.', 'R', '.', '.'],
    ['. ', '.', '.', 'G', '.']
]

rows = len(grid)
cols = len(grid[0])
start = (0,0)
goal = (4,3)

directions = [(0,1),(1,0),(-1,0),(0,-1)]
pq = []
heapq.heappush(pq, (0, start))

visited = set()

while pq:

    cost, current = heapq.heappop(pq)

    if current in visited:
        continue

    visited.add(current)

    if current == goal:
        print("Robot reached Goal")
        print("Minimum Cost =", cost)
        break

    x, y = current

    for dx, dy in directions:

        nx = x + dx
        ny = y + dy

        if (0 <= nx < rows and

```

```

0 <= ny < cols and
grid[nx][ny] != 'X'):

new_cost = cost + 1

if grid[nx][ny] == 'R':
    new_cost += 2

```

output:

Console



Run

```

Run started
Initializing environment
Installing packages
Running code
Robot reached Goal
Minimum Cost = 7
Run completed in 3374.5ms

```

Discussion

Uniform Cost Search evaluates the cumulative cost of every possible path and always expands the least-cost path first. This enables the rescue robot to safely navigate around obstacles, avoid risky zones when beneficial, and adapt to newly discovered information in a partially observable environment.

Conclusion

Using Uniform Cost Search allows the autonomous rescue robot to efficiently reach the survivor while minimizing the total travel cost and satisfying all environmental constraints. It is well suited for dynamic rescue scenarios where safety and optimal decision-making are essential.

```
apq.heappush(pq, (new_cost, (nx, ny)))
```